



Práctica 10

Objetivo

El alumno se familiarizará con el intérprete 80X86 (modo 16bits) sobre la plataforma T-Juino. Esto con el fin de desarrollar programas en lenguaje ensamblador para dicha plataforma.

Introducción

La tarjeta T-Juino es una plataforma de hardware basada en un PCB con un microcontrolador y un entorno de desarrollo, diseñada para facilitar el uso de microcontroladores en proyectos multidisciplinarios.

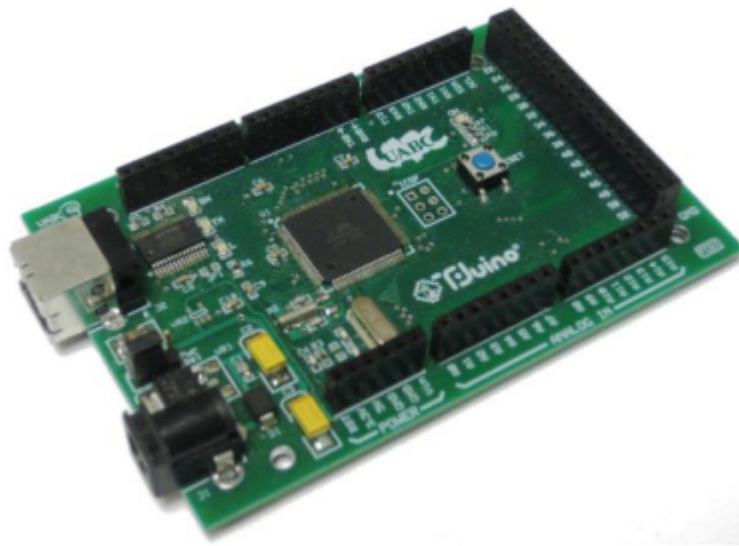


Figura 1. Tarjeta T-Juino.

T-Juino fue diseñado para ser compatible con la plataforma libre llamada Arduino (específicamente Arduino Mega) y el sistema SM8088 de UABC. El hardware consiste de una placa con un microcontrolador Atmel AVR con sus puertos de entrada/salida expuesto mediante conectores. El microcontrolador utilizado en el diseño es el Atmega1280 y para el desarrollo de software se puede utilizar entornos de desarrollo libres como Arduino IDE, Wiring o WinAVR+ Programmer notepad.

Es importante remarcar que una diferencia con la plataforma Arduino es que en T-Juino se cuenta con una máquina virtual (VM88) del procesador 80x86 de 16 bits desarrollada en UABC. Esta máquina lo hace compatible con los programas ejecutables que fueron desarrollados para la plataforma SM8088 basada en el procesador clásico de Intel 80C88.

Desarrollo

Para instalar el intérprete realice los siguientes pasos:

1. Ejecutar la aplicación Xloader, colocar el puerto, seleccionar la tarjeta y cargar el intérprete.

Para utilizar MTTY.

1. Configurar Baud a 19200, paridad none, data bits 8 y stop bit 1.
2. Dar click en conectar.
3. Presionar cualquier tecla al aparecer el símbolo ?.
4. Presionar I y enter para cargar el archivo .com o .bin a cargar.
5. Presionar g para iniciar el programa.
6. Para terminar el programa presionar el push-button de la tarjeta.

Comandos del modo monitor:

Los comandos aceptados en este modo se pueden mostrar mediante la tecla ? que corresponde al comando de help.

Nota: no todos los comandos desplegados están implementados y algunos se encuentran en proceso de su implementación.

Los comandos del modo monitor se muestran a continuación.

*clear	C <range>
dump	D <range>
*enter	E address <list>
*fill	F range list
go	G
help	?
input	I port
load	L
*move	M range address
output	O port value
ports	P
quit	Q
register	R <register value>
trace	T
unassemble	U <range>
version	V
zero (rst)	Z
* - in process	

Comandos

- d** – *dump*: hace un desplegado del contenido de un rango de memoria, el desplegado se hace en formato decimal y ASCII (si el contenido es desplegable).
- g** – *go*: ejecuta el programa que se encuentra cargado en la memoria a partir de la dirección 100h.
- ?** – *help*: muestra la lista de comandos permitidos en el modo monitor.
- i** – *input*: leer un byte de un puerto dado.
- l** – *load*: carga en memoria un archivo (programa) a partir de la dirección 100h.
- o** – *output*: escribe un byte a un puerto dado.
- p** – *ports*: muestra el valor actual de los puertos de entrada-salida (E/S).
- q** – *quit*: sale del modo monitor e inicia el modo de ejecución.
- r** – *registers*: muestra el valor actual de todos los registros del procesador, además puede ser usado para modificar los registro del procesar.
- t** – *trace*: ejecutar la instrucción apuntado por el CS:IP y muestra los valores resultantes de los registros.
- u** – *unassemble*: desensambla un rango de memoria dado mostrando los mnemónicos correspondiente.
- v** – *version*: muestra la versión del interprete que se encuentra cargada en T-Juino.
- z** – *reset*: realiza un reset sobre el intérprete inicializando todos sus registros en cero excepto IP que lo inicializa en 100h.

1) Basándose en el listado 1 crear un archivo texto llamado p10.asm.

Listado 1

```
.model tiny
;----- Insert INCLUDE "filename" directives here
;----- Insert EQU and = equates here

locals

.data
    Mens DB 'Hola Mundo',10,13,0

.code
    org 100h

;*****
; Procedimiento principal
;*****
principal proc
    mov     sp,0fffh           ; inicializar SP (Stack Pointer)
    @@ini0: mov     dx,1
    @@ini1: mov     cx,dx
    @@sigue: mov     al,'x'
             call    putchar
             loop    @@sigue
             mov     al,10
             call    putchar
             mov     al,13
             call    putchar
             inc     dx
             cmp     dx,20
             jbe     @@ini1
             jmp     @@ini0
             ret
    endp
;*****
; procedimientos
;*****
putchar proc
    push    ax
    push    dx
    mov     dl,al
    mov     ah,2               ; imprimir caracter DL
    int     21h               ; usando servicio 2 (ah=2)
    pop     dx                 ; del int 21h
    pop     ax
    ret
    endp

end     principal           ; fin de programa (file)
```

1. Instalar GUI Turbo Assembler v3.0.1 versión 64 bits si es necesario.
2. Ejecutar el programa, abrir la sección de preferencias y modificar los parámetros del linker, eliminar los que están ahí presentes y agregar '/t'.
3. Cargar el archivo .asm
4. Presionar F5 para ensamblar y enlazar el archivo, se genera el archivo .obj y posteriormente el archivo .com.
5. Cargar este último archivo con 'l' en la placa y ejecutarlo con 'g'.

2) Elaborar un diagrama de flujo detallando el proceso para realizar todas las actividades anteriormente presentadas, paso a paso.

3) Implementar **printHex** y **printDecimal** en ensamblador 8086 y correrlo en la tarjeta.

Nota: **printHex** y **printDecimal** sólo deben imprimir las cadenas resultantes, no deben almacenarlas.

Conclusiones y comentarios

Comentar qué diferencias ven los estudiantes con el código de ensamblador del listado 1 a 16 bits con el que ellos conocen de 32 bits

Dificultades en el desarrollo

Referencias